

Informe de Pruebas Unitarias y Cobertura

Proyecto: Tienda Retail Lumina - Arquitectura de Microservicios
Asignatura: Desarrollo FullStack III (DSY1106)
Herramienta de cobertura: JaCoCo 0.8.11
Framework de pruebas: JUnit 5 + Mockito + AssertJ + Spring Boot Test

1. Resumen de cobertura (requisito minimo: 60%)

Componente	Instrucciones	Lineas	Ramas	Cumple >=60%
bff-gateway	76.9%	80.4%	100.0%	SI
ms-usuarios	82.6%	85.7%	60.0%	SI
ms-productos	91.9%	95.8%	100.0%	SI
ms-bodega	89.0%	89.7%	75.0%	SI
ms-carrito	78.8%	90.9%	100.0%	SI
ms-delivery	87.8%	97.0%	100.0%	SI
ms-pagos	89.1%	88.8%	69.2%	SI
TOTAL	85.2%	87.6%	74.7%	SI

Todos los componentes superan el minimo del 60% exigido. La cobertura global de lineas del backend es del 87.6%.

2. Estrategia de pruebas

Las pruebas se organizan por **capa** dentro de cada microservicio:

Tipo	Anotacion / Tecnica	Que valida
------	---------------------	------------

Servicio	@ExtendWith(MockitoExtension.class) + @Mock	Logica de negocio aislada con repositorios mockeados
Controlador	@SpringBootTest + @AutoConfigureMockMvc	Endpoints REST y serializacion
Repositorio	@DataJpaTest + H2 en memoria	Consultas derivadas JPA
Entidad / DTO	JUnit puro	Builders, getters/setters, enums
Excepciones	Test del GlobalExceptionHandler	Manejo centralizado de errores

Ejemplo (ProductoServiceTest)

```
@Test
void obtenerPorId_DebeLanzarExcepcion_CuandoNoExiste() {
    when(productoRepository.findById(999L)).thenReturn(Optional.empty());

    assertThatThrownBy(() -> productoService.obtenerPorId(999L))
        .assertInstanceOf(RuntimeException.class)
        .hasMessageContaining("Producto no encontrado con ID: 999");

    verify(productoRepository).findById(999L);
}
```

3. Cobertura por clase (destacados)

ms-productos (91.9% instrucciones):

- ProductoService : 100%
- ProductoController : 100%
- Producto / Marca (entidades): ~98%

ms-delivery (97.0% lineas):

- DeliveryService , Delivery , HistorialDelivery con alta cobertura.

bff-gateway (76.9% instrucciones):

- Pruebas de los 6 controladores proxy (`@SpringBootTest` + `MockMvc`), validando que cada endpoint exista y enrute correctamente.

4. Como ejecutar las pruebas y generar el reporte

Por microservicio (con Maven instalado)

```
cd ms-productos
mvn test
```

Sin Maven local (usando Docker)

```
docker run --rm -v "${PWD}/ms-productos:/app" -w /app \
maven:3.9-eclipse-temurin-17 mvn test
```

Ubicacion del reporte

Tras ejecutar `mvn test` , el reporte HTML de JaCoCo queda en:

```
<microservicio>/target/site/jacoco/index.html
```

Abrir ese archivo en el navegador muestra la cobertura por paquete, clase y linea con codigo coloreado (verde = cubierto, rojo = no cubierto).

5. Conclusion

El proyecto cuenta con una **bateria de pruebas unitarias e integracion en los 7 componentes backend**, todos por encima del 60% exigido, con una **cobertura global de lineas del 87.6%**. Las pruebas aplican mocking (Mockito) para aislar dependencias y H2 en memoria para validar la capa de persistencia sin requerir infraestructura externa.